

LAB 1: BASIC NEURON MODEL

OBJECTIVES:

- Implement a basic artificial neuron with selectable activation functions (sigmoid, tanh, ReLU) and observe its output on sample inputs.
- Visualize and compare common activation functions.
- Build a simple 3-layer feedforward neural network and perform a forward pass on random input to inspect outputs and shapes.

THEORY:

An artificial neuron computes a weighted sum of inputs and adds a bias, $z = w^T x + b$, then applies an activation function to produce the output. Activation introduces non-linearity, enabling the network to model complex patterns.

This lab uses sigmoid, tanh, and ReLU activations. Sigmoid outputs values in (0,1) but can cause vanishing gradients. Tanh outputs values in (-1,1) and converges faster due to zero-centering. ReLU outputs $\max(0, x)$, reducing gradient issues but may create dead neurons. Sigmoid inputs are clipped for numerical stability.

A single hidden layer feedforward network is used. Forward propagation computes $Z_1 = XW_1 + b_1$, $A_1 = \text{sigmoid}(Z_1)$, then $Z_2 = A_1W_2 + b_2$ to produce the output. A_2 . Weights are initialized with small random values and biases are set to zero. The lab focuses only on forward propagation, without training or backpropagation.

Algorithm:

For Simple Perceptron:

1. Start
2. Initialize parameters
3. `BasicNeuron.forward(inputs)`: compute $z = \text{inputs} \cdot \text{weights} + \text{bias}$; apply activation f according to the selected mode (sigmoid with clipping, tanh, or ReLU); return $f(z)$ (or z if no activation).
4. `SimpleNetwork.forward(X)`: compute $Z_1 = XW_1 + b_1$, $A_1 = \text{sigmoid}(Z_1)$, $Z_2 = A_1W_2 + b_2$, $A_2 = \text{sigmoid}(Z_2)$; return A_2 .
5. `Plot_activation_functions()`: generate x over $[-5, 5]$, compute curves for sigmoid, tanh, and ReLU, and render them on separate subplots.
6. End

SOURCE CODE:

```
import numpy as np
import matplotlib.pyplot as plt

class SimpleNeuron:
    def __init__(self, n_in, mode='sigmoid'):
        self.w = np.random.randn(n_in) * 0.5
        self.b = np.random.rand() * 0.5
        self.mode = mode

    def activate(self, x):
        if self.mode == 'sigmoid':
            return 1 / (1 + np.exp(-np.clip(x, -250, 250)))
        elif self.mode == 'tanh':
            return np.tanh(x)
        elif self.mode == 'relu':
            return np.maximum(0, x)
        return x
    def forward(self, x):
        z = np.dot(x, self.w) + self.b
        return self.activate(z)

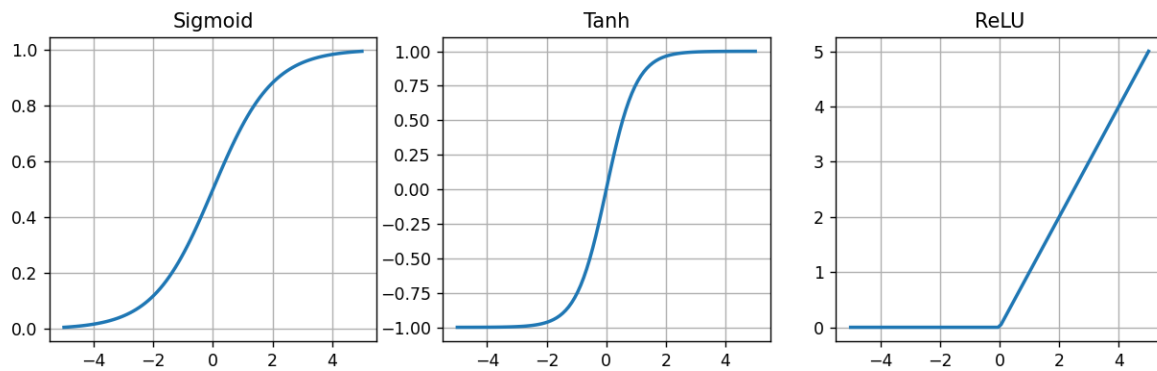
class NeuralNet:
    def __init__(self, n_in, n_hid, n_out):
        self.w1 = np.random.randn(n_in, n_hid) * 0.5
        self.b1 = np.zeros((1, n_hid))
        self.w2 = np.random.randn(n_hid, n_out) * 0.5
        self.b2 = np.zeros((1, n_out))
    def run(self, x):
        h_out = 1 / (1 + np.exp(-np.clip(np.dot(x, self.w1) + self.b1, -250, 250)))
        y_out = 1 / (1 + np.exp(-np.clip(np.dot(h_out, self.w2) + self.b2, -250, 250)))
        return y_out
    def plot_functions():
        x = np.linspace(-5, 5, 100)
        funcs = [1/(1+np.exp(-x)), np.tanh(x), np.maximum(0, x)]
        names = ["Sigmoid", "Tanh", "ReLU"]

        plt.figure(figsize=(12, 3))
        for i in range(3):
            plt.subplot(1, 3, i+1)
            plt.plot(x, funcs[i], linewidth=2)
            plt.title(names[i])
            plt.grid(True)
        plt.show()

node = SimpleNeuron(3, mode='sigmoid')
print(f'Neuron Output: {node.forward(np.array([0.5, -0.2, 0.8])):.4f}')

net = NeuralNet(2, 4, 1)
print(f'Network Shape: {net.run(np.random.randn(5, 2)).shape}')
plot_functions()
```

OUTPUTS:



Neuron Output: 0.5607

Network Shape: (5, 1)

CONCLUSION:

This experiment demonstrates forward propagation in a single neuron and a simple feedforward neural network using affine transformations followed by nonlinear activation functions. The activation plots show that sigmoid and tanh saturate at large input values, while ReLU remains piecewise linear with zero output for negative inputs. The forward pass produces outputs with correct dimensions, such as (5, 1) for a 5×2 input batch, confirming proper vectorized computation. Since weights are randomly initialized and no training is performed, the outputs are not predictive, but the lab clearly illustrates activation behavior, numerical stability considerations, and the effect of activation functions on signal flow.